Complex Wavelet Transform for time-frequency decomposition

TUTORIAL IIB

Tutor: Stefan Ehrlich (stefan.kh.ehrlich@gmail.com)

Synopsis

Complex Wavelet Transform (CWT) is one of the most established methods for time-frequency decomposition. In the field of EEG research CWT is frequently used to elaborate the standard ERP technique and has proven highly useful for the analysis of oscillatory activity. Importantly, CWT is not constrained to EEG signals only: CWT is used in many other domains, such as the electrocardiogram or electromyography signals, but also non biomedical domains, such as image compression.

Many high-quality in-built functions for CWT are available and can be used with a single click or one line of code implementation. Oftentimes, researchers use these in-built functions without properly understanding the underlying mechanisms, which can lead to poor choices of parameters and questionable interpretation of the results.

The goal of this tutorial is to manually implement the CWT method and apply it to dummy data, as such to learn and get a feeling about the mechanisms of CWT before applying it to real EEG data.

1: Convolution in the time- and frequency domain

Load the dummy data (*dummyX.mat*) into your Matlab workspace and explore its content. Make yourself familiar with the data *dummyX.x* (e.g. plot it) and visualize the kernel *dummyX.gausskernel*. This part of the tutorial is meant to practice convolution using time- and frequency domain methods.

1a: convolution via MATLAB `conv` function:

Use the MATLAB function `conv` to obtain the convolution results. The length of your conv-results is N+K-1 (N: length of signal, K: length of kernel). Prune the *conv-result* to appropriately match the length of your original signal.

1b: manual convolution via piece-wise dot product

Implement a for-loop and convolve the kernel manually with the signal by computing the sample-by-sample dot product. Do not forget to zero-pad your signal at the beginning and the end to match the appropriate length N+K-1. After convolution, prune the *conv-result* to match the length of your original signal.

$$dotproduct_{ab} = \sum_{i=1}^{K} sig_i * kernel_i$$

1c: FFT-based convolution

The convolution theorem describes the equivalence of time-domain convolution with frequency-domain multiplication. To get a sense for this, (a) compute the FFT separately for the signal and the kernel; (b) multiply the resulting complex spectra; and (c) apply the inverse FFT (`ifft`) to the multiplied spectrum, this way to obtain the *conv-result*. Do not forget zero-padding before FFT-decomposition: importantly here, you need to zero-pad both the signal and the kernel to match the appropriate length N+K-1. Afterwards, prune the *conv-result* to match the length of your original signal.

Finally, plot all three *conv-results* in a single plot (use MATLAB command `hold on`) and double check if they are identical. If not, use *conv-result* from section 1a as a reference to fix your code.

2: Creating a single complex Morlet wavelet (CMW)

Create a complex Morlet wavelet of equal sampling rate as the signal. The wavelet should have a length of *1 s*, a frequency of *f=10Hz*, and *nC=5* number of cycles. Start out by (a) creating a zero-centered time-vector which starts from -0.5 s and ends at 0.5 s. Then, use the equations below to implement (b) the complex sine wave, (c) the gaussian window, and then (d) the resulting complex Morlet wavelet.

$$complex_sine = e^{i2\pi ft}$$

$$gauss_win = e^{-t^2/2s^2}$$

$$cmw = e^{-t^2/2s^2} e^{i2\pi ft}$$

$$s = \frac{nC}{2\pi f}$$

Create a superimposed plot showing the real-part of the complex sine wave, the gaussian window and the real-part of the CMW (use MATLAB function `real`, to extract the real-part).

Create a 3d-plot (MATLAB function `plot3`) showing the entire CMW including real- and imaginary parts. Explore the wavelet by rotating the 3d plot.

3: Apply CMW to dummy data and compute CWT results

Now, apply your CMW to the signal by using the FFT-based convolution (see section 1c). In order to not add any power from the wavelet kernel into your wavelet transformation, make sure you normalize the FFT-spectrum of your CMW (*cmw./max(cmw)*) before you multiply it with the FFT of your signal. As already described in section 1c, do not forget to zero-pad both signal and kernel before FFT-decomposition. Prune the resulting wavelet transform to appropriately match the length of your original signal.

Compute and visualize (a) the sample-by-sample band-filtered signal, (b) the sample-by-sample magnitude, (c) the sample-by-sample power and (d) the sample-by-sample phase diagram. The corresponding equations can be found in the lecture slides.

4: Create a family of CMWs and apply to dummy data

Now, create a *family* of complex Morlet wavelets ranging linearly from 4 Hz to 250 Hz in steps of 2 Hz. Also, linearly vary the number of cycles starting from *nC=3* cycles at the lowest frequency and ending at *nC=10* cycles at the highest frequency. This operation should give you 124 individual wavelets. Hint: use the MATLAB function `linspace` to create linearly spaced vectors.

Apply each of these wavelets one after the other to your signal and store the resulting wavelet transformation results.

Compute and visualize the time-frequency plot of the wavelet magnitude. For visualization, use the MATLAB function `contourf` with an interpolation factor of 10 and without contour lines (`'linecolor','none'`).

QB-4 (15 points): Place the time-frequency visualization in your report and describe it in your own words. Identify the missing temporally stationary sine-wave components in the signal - both in terms of the time-period in which they are present and their frequencies. Can you improve the time-frequency decomposition by changing some of the parameters above, e.g. the range of frequencies or the range of number of cycles? Explore alternatives and describe the best suitable set of parameters you could find.

References / further reading:

[1] Cohen, M. X. (2014). *Analyzing neural time series data: theory and practice*. MIT press.

[2] http://mikexcohen.com/lectures.html